📘 Overview of Git

🔹 What is Git?

Git is a **distributed version control system (DVCS)** used to track changes in source code during software development. It allows multiple developers to work on a project simultaneously without interfering with each other's work.

It was created by Linus Torvalds in 2005 to manage the development of the Linux kernel.

🔹 Why Git is Important

From a DevOps and software engineering perspective, Git is foundational because it:

- Maintains **history of code changes**
- Enables **collaboration across teams**
- Supports **branching and merging strategies**
- Acts as the backbone for **CI/CD pipelines**
- Provides **traceability and rollback capabilities**

🔹 Key Concepts in Git

1. Repository (Repo)

A **repository** is where your project lives. It contains:

- Source code
- Commit history
- Branches

Types:

- **Local repository** (on your system)
- **Remote repository** (hosted platforms like GitHub, GitLab)

---

2. Commit

A **commit** is a snapshot of your code at a specific point in time.

- Each commit has a unique ID (SHA hash)
- Includes message, author, timestamp

---

3. Branch

A **branch** is a parallel version of your codebase.

- Default branch: main (or master)
- Used for:
    - Feature development
    - Bug fixes
    - Experiments

---

4. Merge

Combining changes from one branch into another.

Example:

- feature/login → merged into main

---

5. Clone

Creating a copy of a remote repository on your local machine.

---

6. Pull & Push

- **Pull** → Fetch and merge changes from remote to local
- **Push** → Send local commits to remote repository

🔹 Git Architecture (How It Works Internally)

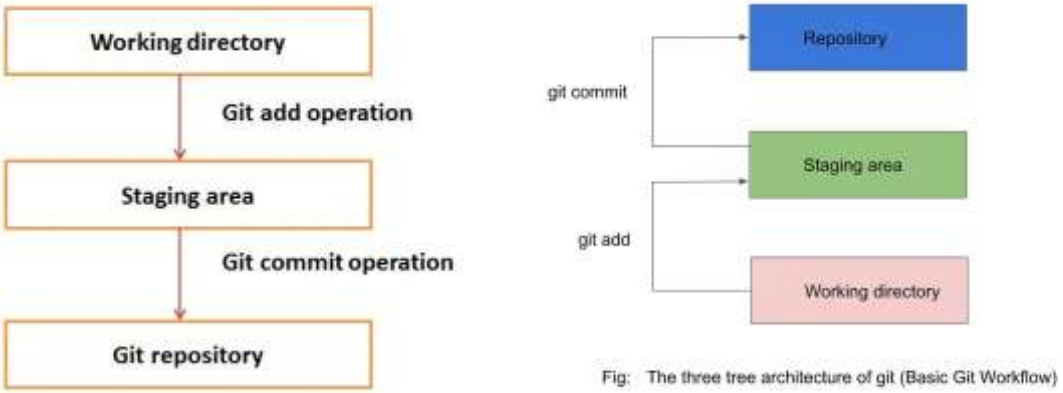


Fig: The three tree architecture of git (Basic Git Workflow)

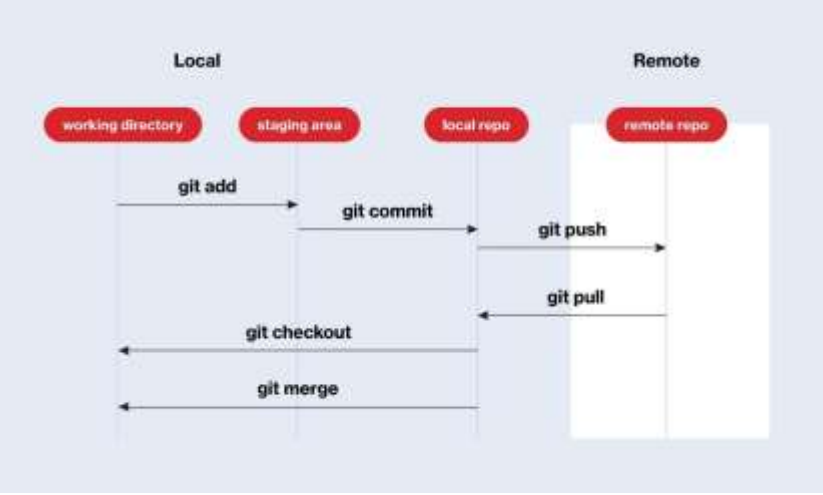
Git has three main areas:

1. **Working Directory**
    - Your actual project files
2. **Staging Area (Index)**
    - Where changes are prepared before commit
3. **Local Repository**
    - Stores committed snapshots

👉 Flow:

Working Directory → Staging Area → Repository

(git add) (git commit)



- **Basic Git Workflow**

  Initialize repository:

  git init

  Add files:

  git add .

  Commit changes:

  git commit -m "Initial commit"

  Connect to remote:

  git remote add origin <url>

  Push code:

  git push origin main

- **Centralized vs Distributed Version Control**

Feature	**Centralized (e.g., SVN)**	**Git (Distributed)**
Repository	Single central server	Every developer has full copy
Offline Work	Limited	Fully supported
Speed	Slower	Faster
Branching	Complex	Lightweight & fast

- **Advantages of Git**
  - ✓ Fast and efficient
  - ✓ Strong branching model
  - ✓ Offline capabilities

- ✓ Distributed collaboration
- ✓ Widely adopted in DevOps pipelines

🔹 **Git in DevOps Lifecycle**

Git integrates with:

- Build tools (Maven, npm)
- CI/CD tools (Jenkins, Azure DevOps)
- Containerization (Docker)

👉 Example flow:

Developer → Git Commit → CI Pipeline → Build → Test → Deploy

🔹 **Common Use Cases**

- Source code management
- Version tracking
- Team collaboration
- Open-source contributions

🔹 **Summary**

Git is not just a tool—it is a **core skill** for:

- Developers
- DevOps Engineers
- QA Engineers

It provides **control, collaboration, and confidence** in managing codebases.
